



OASIS Security Report

Insecure Cryptographic Usage Security Analysis

Date: 2026-06-11 09:55:35 — **Model:** mistral:latest

Vulnerability: Insecure Cryptographic Usage

Project: clean_contracts

Analysis root: `../clean`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for Insecure Cryptographic Usage.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical

(

0

High		0
Medium		1
Low		1

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.628
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.625

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.628

Source lines: 1-42

► Finding 1.1: Insecure Cryptographic Usage (Medium)

Vulnerable code

```
keccak256(abi.encodePacked("Nu Token")) == bank.supportsToken(),
```

Explanation

The 'keccak256' hash function should not be used as a comparison function to validate contract interactions.

Impact

Data breaches or unauthorized function calls if the contract is compromised.

Execution path

```
contract -> supportsToken()
```

Exploitation steps

- Compromise the contract to call `supportsToken`
- Replace 'Nu Token' with a different token and return true

Example payloads

- `attacker_token`

Conditions

A malicious actor able to compromise the contract.

Remediation

Use a secure comparison method for validation, such as '==' when comparing strings or use ECDSA keys if possible.

Secure example

```
require(abi.encodePacked("Nu Token") ==  
abi.encodePacked(bank.supportsToken()),)
```

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.625

Source lines: 1-34

► Finding 2.1: Vulnerability found (Low)

Vulnerable code

```
'pragma solidity 0.4.24;', 'function donate(address to) payable public{
    credit[to] += msg.value;
}', 'function withdraw(uint amount) public {
    if (credit[msg.sender]>= amount) {
        credit[msg.sender]-=amount;
        require(msg.sender.call.value(amount)());
    }
}'
```

Explanation

The code uses Ethereum's Solidity language without any cryptographic algorithms or hash functions, hence there are no insecure implementations of these to analyze.

Impact

No confidentiality breaches as the contract does not handle sensitive data directly.

Entry point

Solidity contract

Execution path

```
'''
Solidity Contract
|_
|_
Functions: donate, withdraw
'''
```

Exploitation steps

- Interact with the smart contract functions

Conditions

None

Remediation

Review usage of cryptographic algorithms and hash functions within the Solidity contract.

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)